

Benchmarking and Performance Analysis of LLM Inferencing Systems

Saksham Rathi

Systems for LLMs
Under Prof. Mythili Vutukuru

Indian Institute of Technology Bombay
Autumn 2025-26

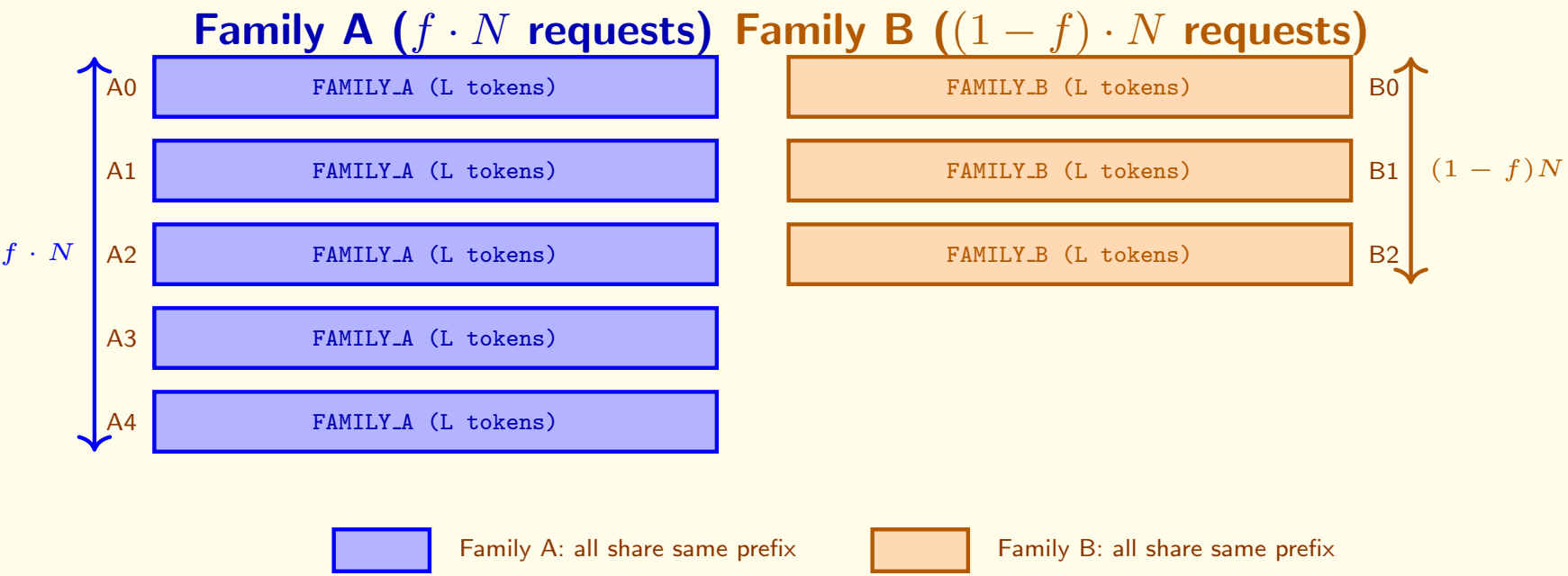
Contents

1. Motivation
2. Initial Experiments
3. Key Idea
4. Chunked Hash Tree
5. RL Decision Maker
6. Results

Motivation

- Shared prefix workloads
- Targeting both **multi-family** and **radix-tree** style workloads
- No one so far has analyzed **DRAM** bandwidth utilization and **L2** cache hit rates
- No one cared about **temporal** and **spatial** locality, while accessing the KV blocks
- So far the focus has been on HBM space utilization
- Lack of CPU-efficient schedulers that work for dynamic workloads

Two-family batch mixture



- Two large prefixes
- A fraction f of the requests in a batch belong to family A, and the remaining requests belong to family B

Systems for LLMs
Saksham
Motivation
Initial Experiments
Key Idea
Chunked Hash Tree
RL Decision Maker
Results

Two Family Batch Mixture (L = 2000 tokens)

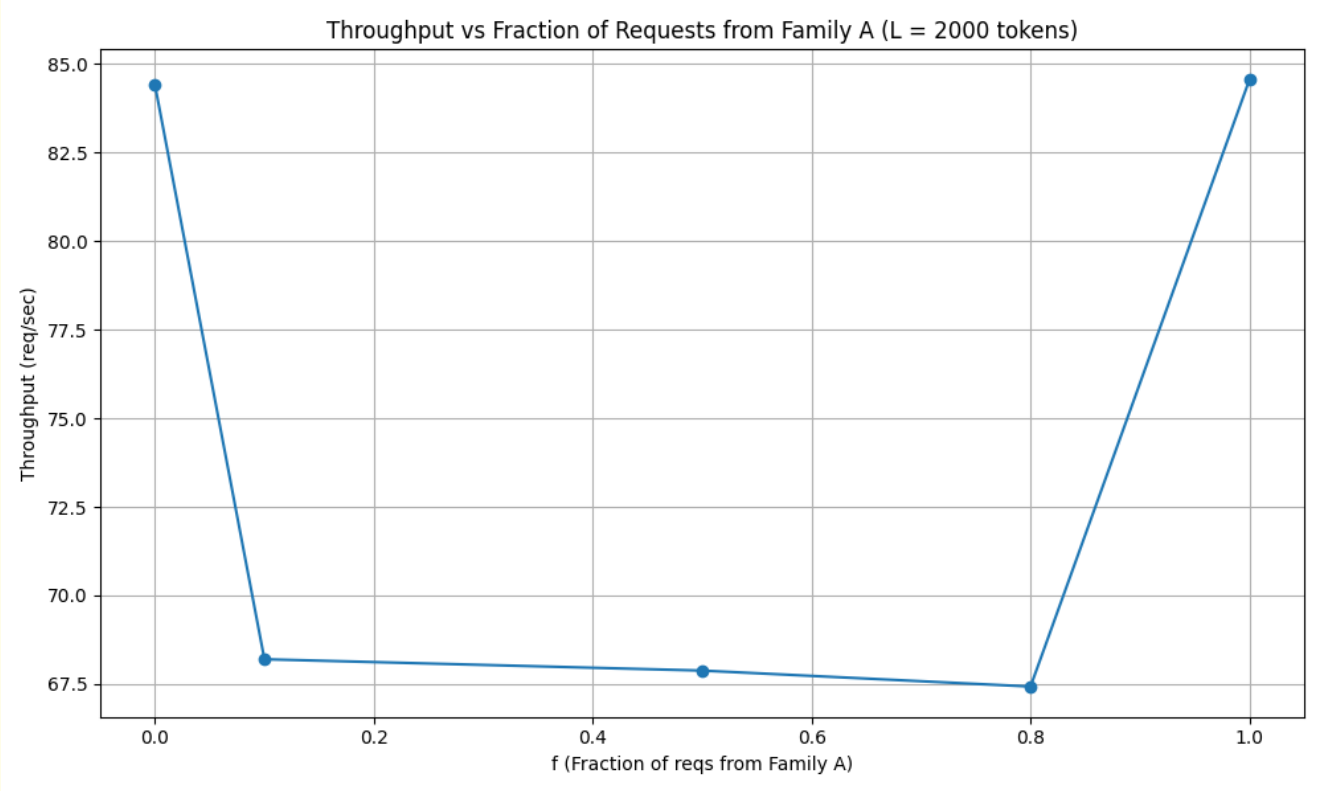


Figure: Results across fraction of requests from A

- All the prefill has already been completed so far
- Both the families are 2000 tokens in size
- Stark difference in throughput numbers between homogeneous and even slightly heterogeneous batches

Two Family Mixture (L = 2000 tokens) - Mean DRAM Activity

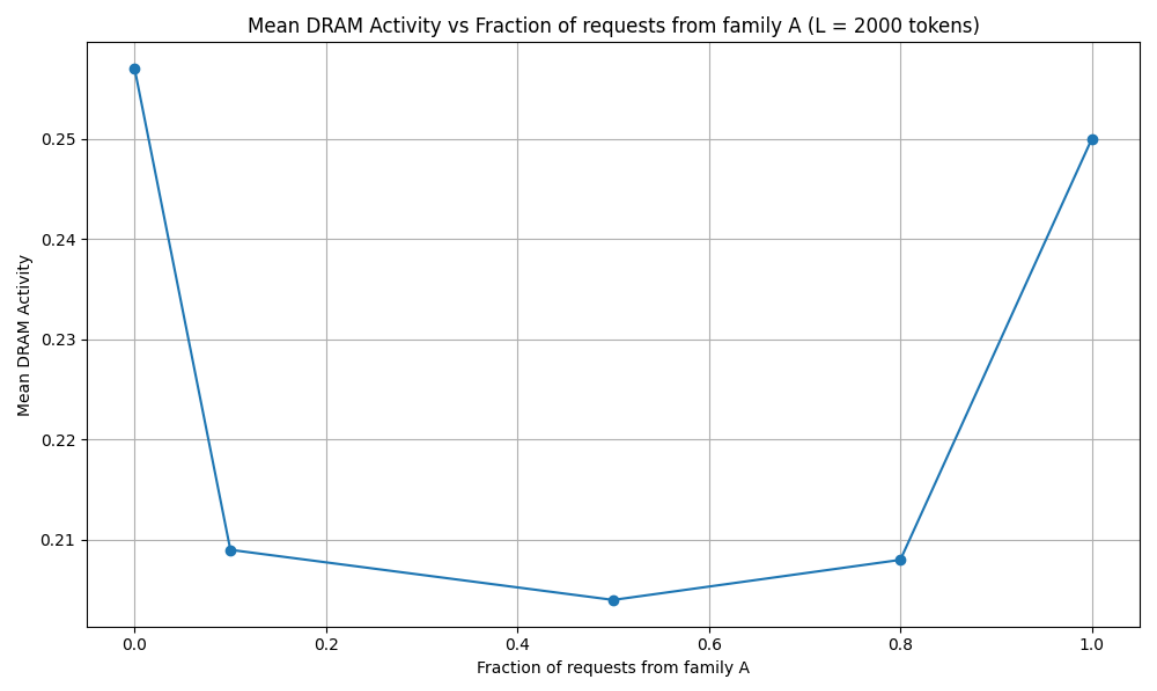


Figure: Mean DRAM Activity across fraction of requests from A (L = 2000 tokens)

- DRAM activity higher for homogeneous batches
- Sequential accesses → more spatial locality → higher prefetcher confidence → more requests issued → higher DRAM utilization

Two Family Mixture (L = 2000 tokens) - Total DRAM Accesses

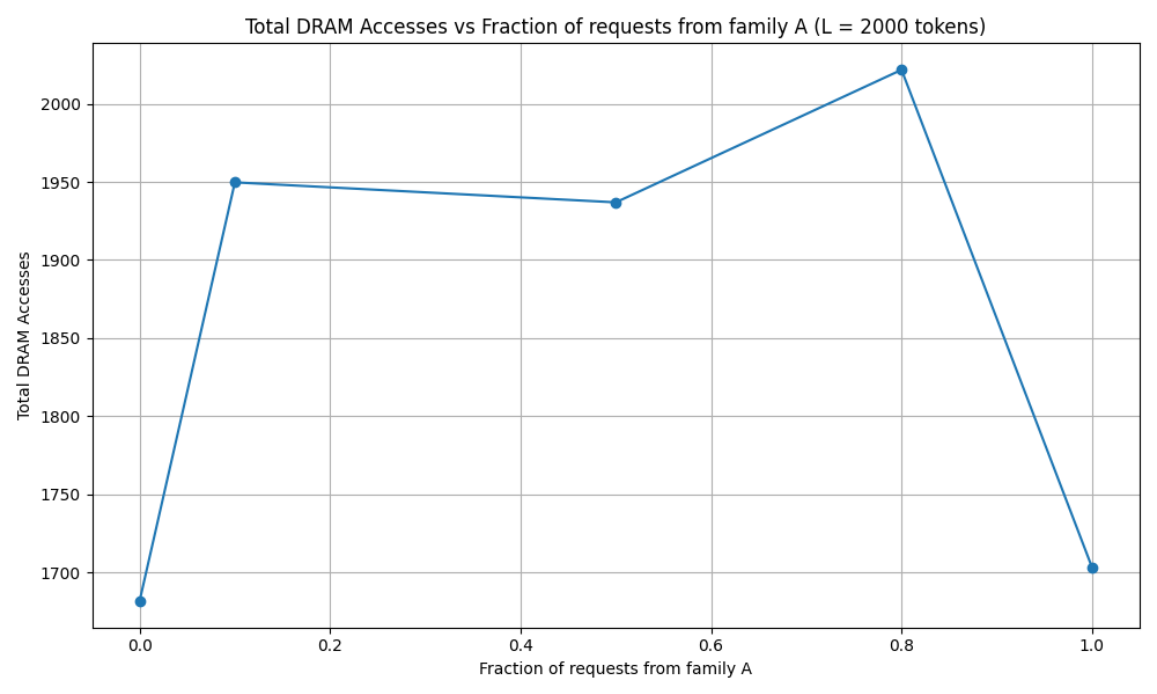
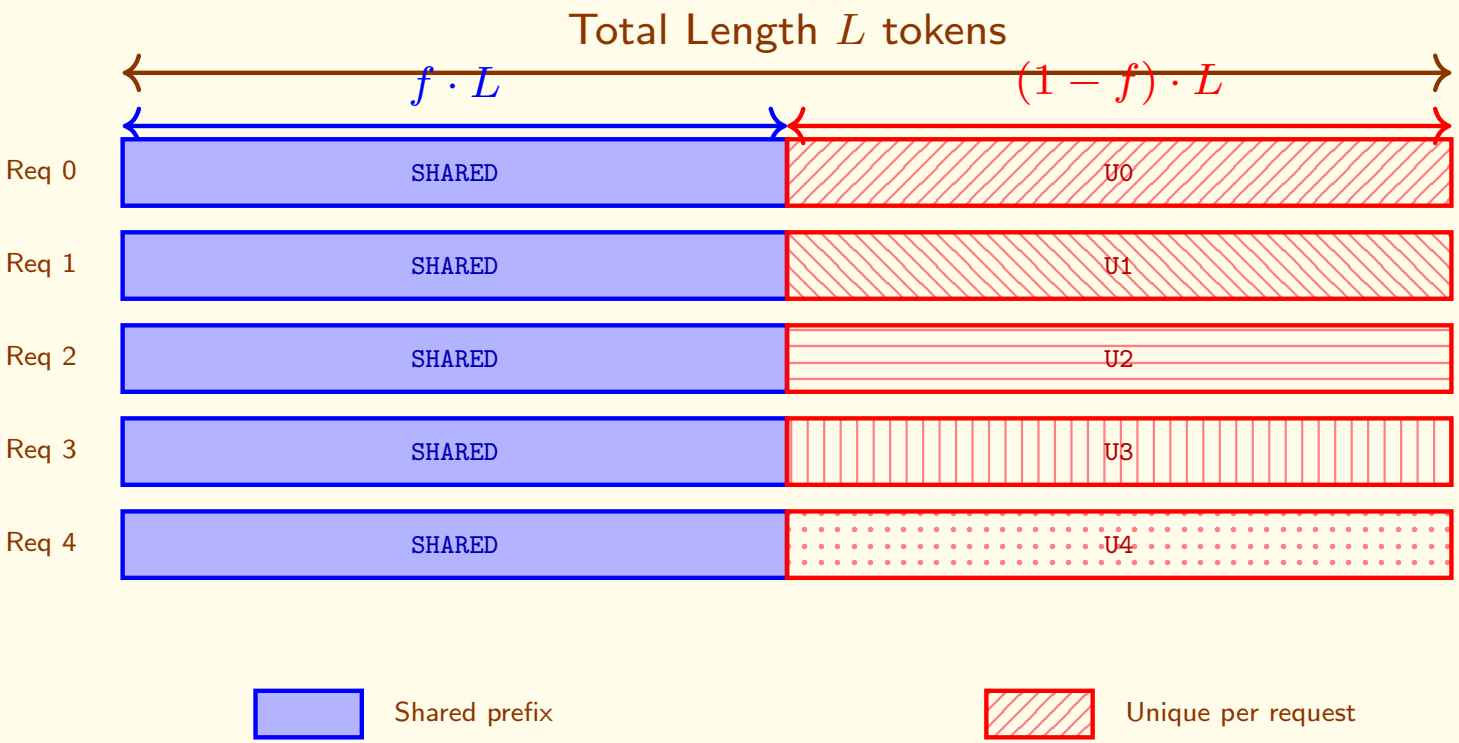


Figure: Total DRAM Accesses across fraction of requests from A (L = 2000 tokens)

- Total number of DRAM accesses lower for homogeneous batches
- Requests running in parallel → higher number of requests sharing the same block → better temporal locality → larger cache hit rates → less DRAM accesses

Systems for LLMs
Saksham
Motivation
Initial Experiments
Key Idea
Chunked Hash Tree
RL Decision Maker
Results

Varying the amount of sharing



Varying the amount of sharing (L = 500 tokens)

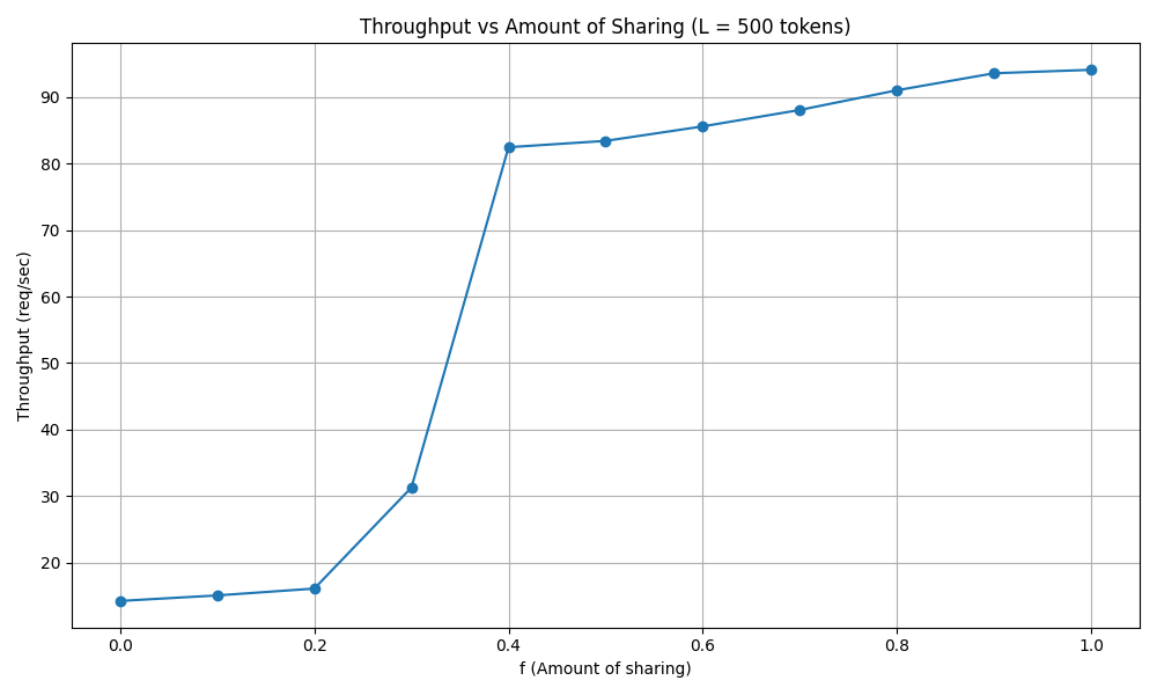


Figure: Results across varying L

- Initial throughput numbers quite low, because of lack of space in HBM
- f increases $\rightarrow$ throughput increases
- A larger chunk shared $\rightarrow$ a larger duration of spatial and temporal locality

Key Conclusions from the initial experiments



- Memory requests from different requests are roughly simultaneous
- Sequential accesses are quite helpful to utilize DRAM bandwidth fully
- Enough requests should access nearby blocks to ensure high cache hit rates
- Batches should be completely homogeneous to utilize these benefits fully
- Merging requests from the same prefix together can also help in efficient KV cache eviction from the HBM $\rightarrow$ avoiding recomputation

Key Ideas and Metric Proposed



Metric:

- (Number of requests running) $\times$ (Number of prefix chunks shared by **all** of them)
- Higher number of requests $\rightarrow$ efficient GPU utilization
- More number of prefix chunks shared $\rightarrow$ larger duration of spatial and temporal locality
- All requests $\rightarrow$ heterogeneous batches do not help

Key Ideas:

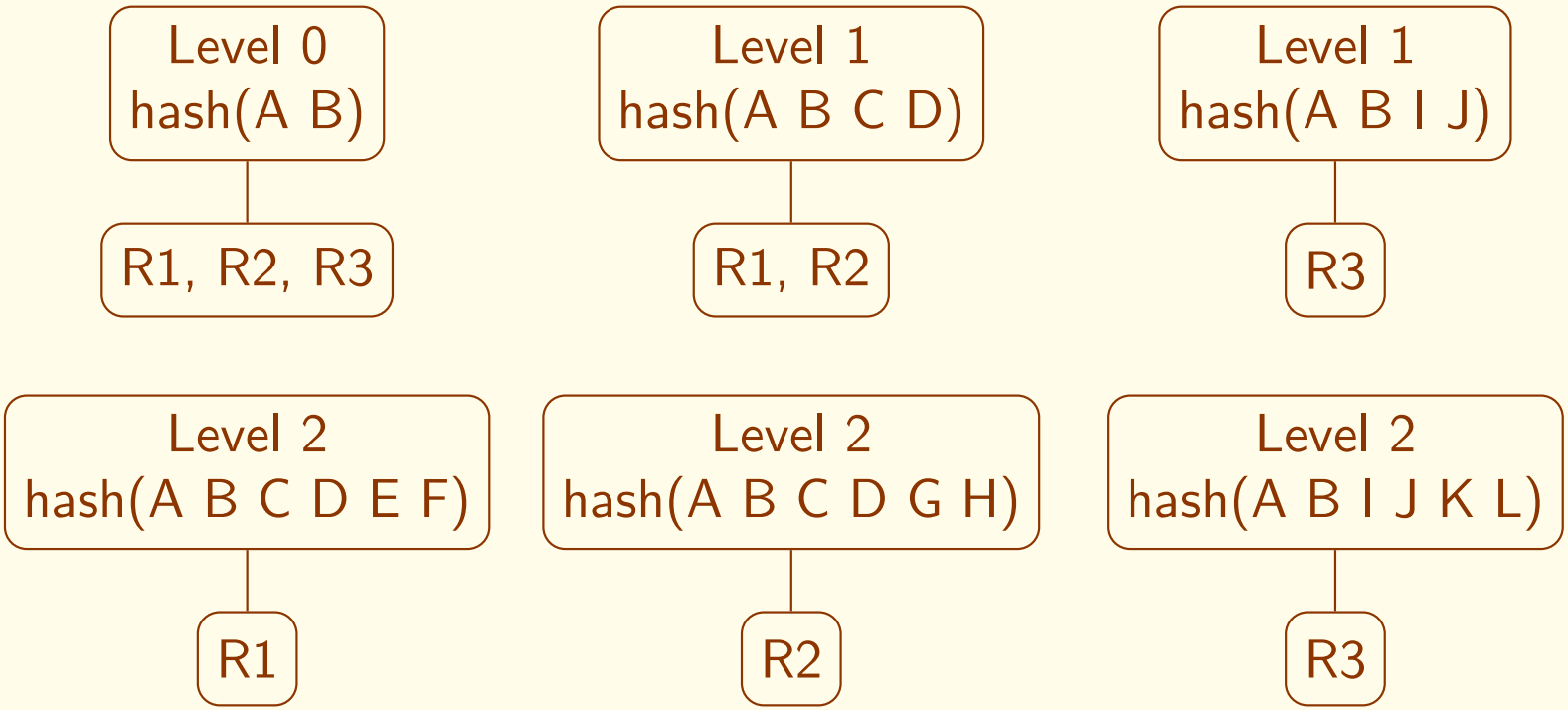
- Efficiently compute shared chunks $\rightarrow$ **Chunked Hash Tree**
- Decision on when to stop the batch creation process $\rightarrow$ **RL**

Chunked Hash Tree



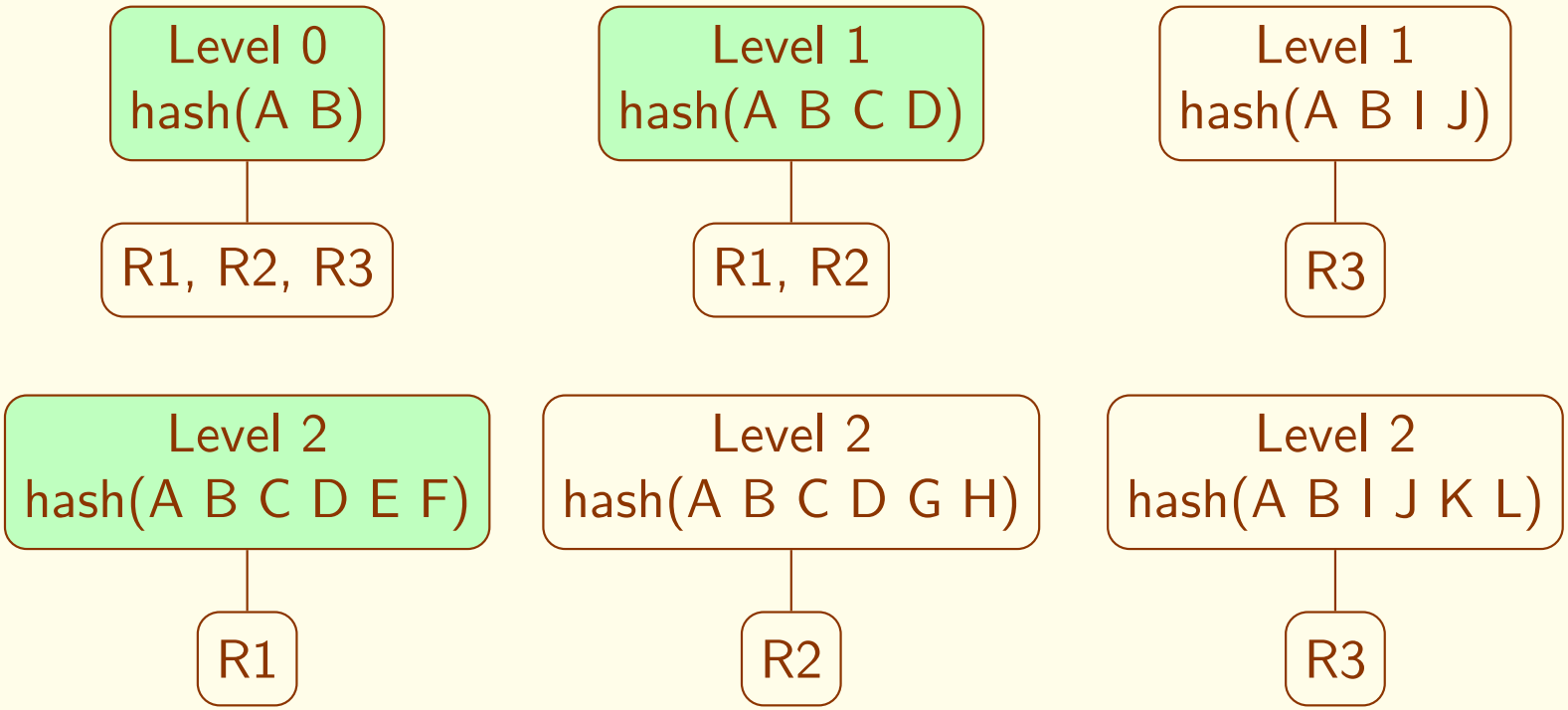
- Radix-tree selection is slow for large, online, long-context workloads
- Requests are represented using **hashed cumulative prefixes**
- Tokens are divided into fixed-size chunks
- Each level corresponds to `hash(tokens[0:k])`
- No explicit tree structure or node splitting
- Much faster than radix trees for large workloads
- Working set = set of active **(level, hash)** pairs
- Scheduling prefers requests with minimal missing chunks

Chunked Hash Tree - Insertion



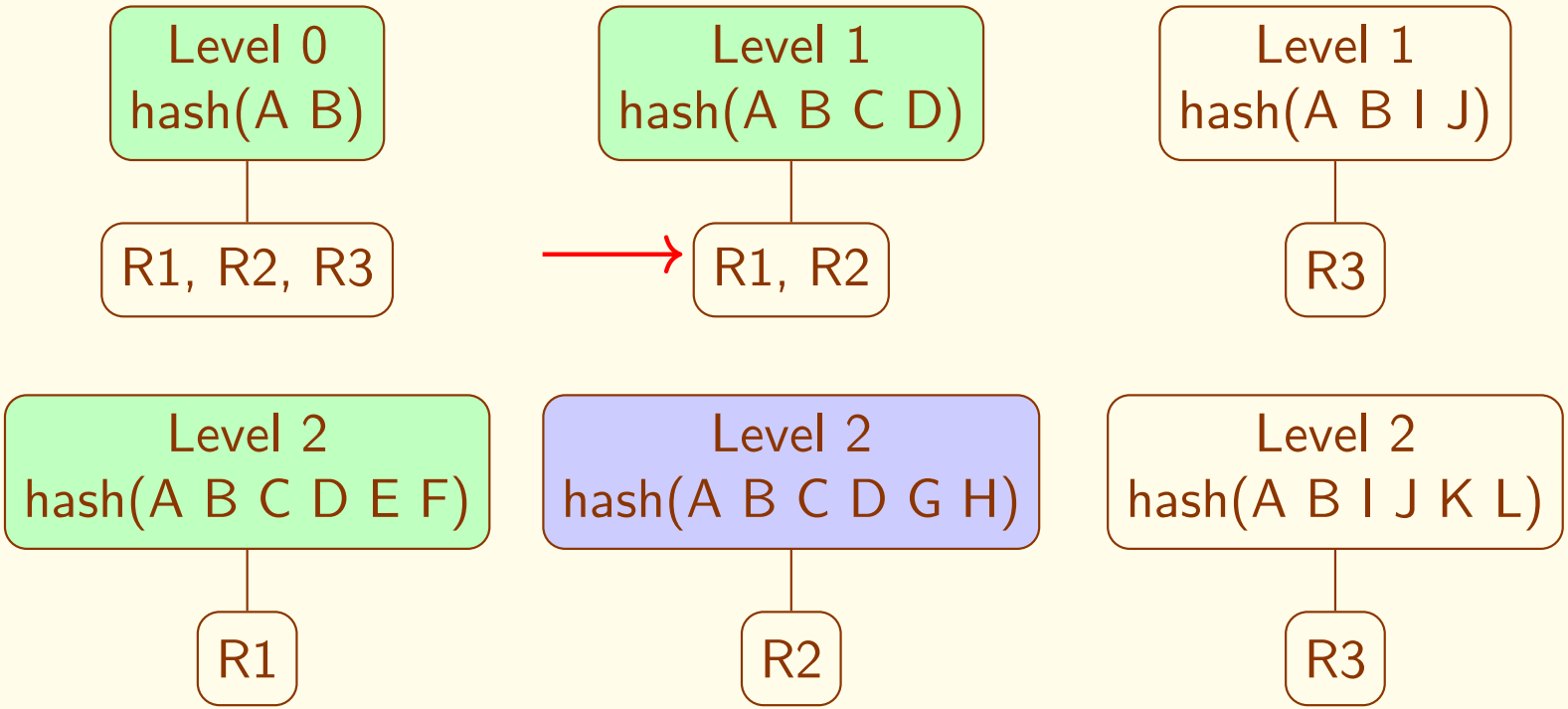
Each level stores hashes of cumulative token prefixes.

Chunked Hash Tree - Working Set



Green boxes indicate prefix hashes in the working set.

Chunked Hash Tree - Best Request Selection



Green boxes indicate prefix hashes in the working set.

R2 is selected since it has maximum prefix overlap with the working set.

Formulation

State:

- **B:** current batch size
- $\mathbf{C_B}$: number of chunks shared before adding a new request
- $\mathbf{C_A}$: number of chunks which will remain shared after adding the new (best) request
- $\mathbf{R_W}$: number of requests waiting at the end of C_A

Action:

- **ADD:** pop and activate the best request from the chunked hash tree
- **SKIP:** break the batch construction loop, dispatch the batch to GPU

Reward:

- Negative decode wall step time of the previous batch
- Delivered once per batch

Policies

- **Q Learning:** The system tries actions (ADD or SKIP), observes rewards and gradually learns which action works best in each situation with respect to the long term outcome.
- **Contextual Bandit:** Which action works best right now (ignoring long-term future) in this situation?
- **Heuristics:** Based on current batch sizes and change in the number of chunks

Throughput vs Req Rate (5 families) (Sequence Length: 3900 tokens)

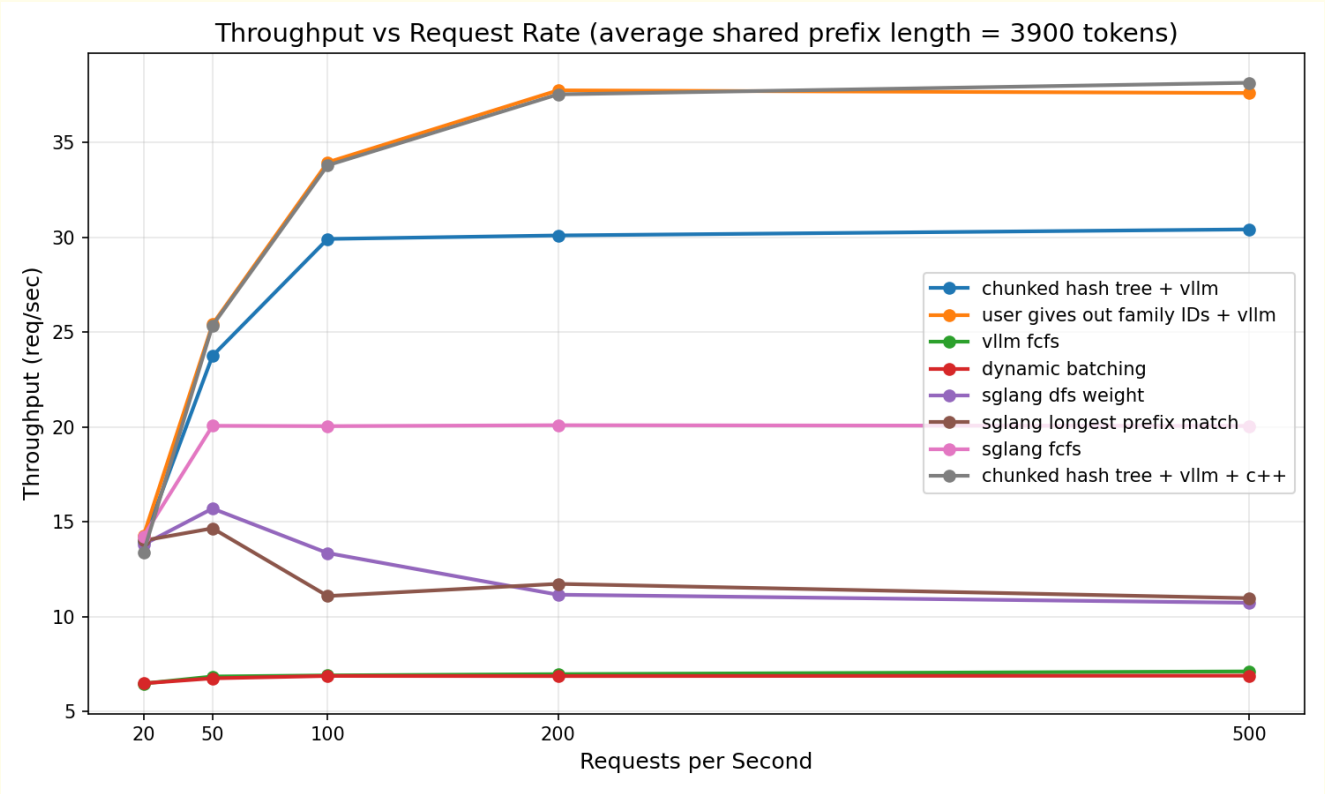


Figure: Throughput vs Request Rate (5 families)

Throughput vs Req Rate (5 families) (Sequence Length: 13000 tokens)

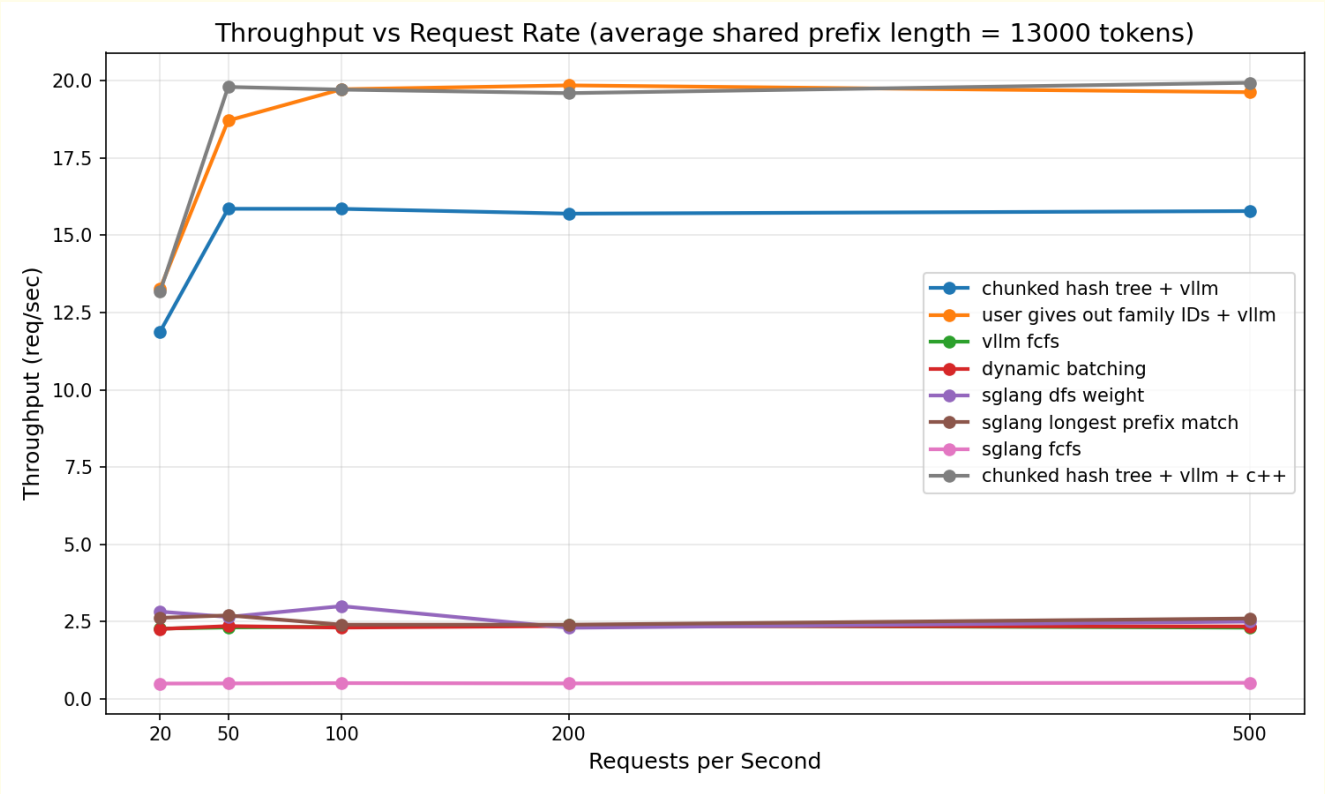


Figure: Throughput vs Request Rate (5 families)



Thank You